

# Diseño e implementación de un sistema basado en Servicios Web con Python (FastAPI)

Amjad A Habeeb Mahhi  
Mouad Harrouch Boukhalfa

## Caso 14: Protocolo de votación electrónica con control de unicidad y cierre formal

### 1. Transformación de Métodos a Recursos REST

La arquitectura REST (Representational State Transfer) impone un cambio de paradigma: pasamos de invocar procedimientos remotos (RPC) a manipular el estado de recursos identificables a través de una URL mediante un conjunto uniforme de métodos HTTP.

La siguiente tabla ilustra la correspondencia exacta aplicada:

| Funcionalidad Original     | Enfoque RESTful          | Endpoint       | Payload                                   | Responses (HTTP Codes)           |
|----------------------------|--------------------------|----------------|-------------------------------------------|----------------------------------|
| emitirVoto(dni, candidato) | Creación de entidad Voto | POST<br>/votos | {"dni": "...", "candidato": "..."}<br>... | 201 (Creado), 400, 403, 404, 409 |
| cerrarUrna()               | Actualización parcial    | PATCH<br>/urna | {"estado": "cerrada"}<br>...              | 200 (OK), 409 (Conflicto)        |

En un diseño orientado a RPC, se utilizaría una URL como `/votar` o `/cerrar-urna`, lo cual es un claro antipatrón (RPC disfrazado). El diseño implementado respeta a las entidades (sustantivos `votos` y `urna`) delegando la acción al verbo HTTP correspondiente (POST para añadir, PATCH para modificar propiedades).

### 2. Diferencias respecto a Objetos Distribuidos (Java RMI / Sockets)

- **Manejo de Excepciones y Errores:** En RMI, el flujo normal y los errores se solapaban, bien devolviendo cadenas de error tipificadas (ej. `"dni_invalido"`) o lanzando `RemoteException`. REST aprovecha los códigos de estado HTTP para separar la metadata del error de su payload. Un `409 Conflict` comunica inmediatamente el rechazo de un duplicado sin necesidad de parsear el cuerpo del mensaje.
- **Acoplamiento Cliente-Servidor:** RMI requiere que el cliente y el servidor compartan interfaces o "Stubs" estrechamente acoplados al lenguaje (Java). Con sockets TCP rudimentarios, el servidor exigía un ABNF exacto sin tolerancia a fallos de serialización. FastAPI, empleando REST y JSON sobre HTTP, provee desacoplamiento tecnológico absoluto. El cliente ahora puede ser web, móvil o script, basándose solo en el contrato OpenAPI.
- **Negociación de Protocolo:** En el antiguo sistema con sockets había que gestionar búferes (`recv(1024)`), codificaciones (`utf-8`), saltos de línea y fragmentación. En HTTP, el servidor web ASGI (Uvicorn) abstrae por completo el protocolo subyacente de transporte.

### 3. Ventajas e Inconvenientes de REST

#### Ventajas:

- **Interoperabilidad Universal:** La combinación de HTTP y JSON es un estándar de facto que cualquier framework moderno comprende.
- **Documentación Viva:** Mediante OpenAPI, las APIs REST pueden autogenerar portales interactivos (Swagger/ReDoc), facilitando su consumo sin requerir manuales externos.
- **Escalabilidad y Caché:** Al usar verbos HTTP correctamente, es sencillo integrar balanceadores de carga o mecanismos de caché (aunque en este caso POST y PATCH no sean cacheables, es fundamental para futuras extensiones como GET).

`/urna`).

#### Inconvenientes:

- **Sobrecarga (Overhead):** A diferencia de un socket puro o RMI binario, HTTP introduce cabeceras, texto plano y pasos de serialización JSON en cada petición que pueden perjudicar la latencia en entornos de ultra-baja latencia.
- **Unidireccionalidad:** HTTP clásico (sin Server-Sent Events o WebSockets) obliga al cliente a hacer "polling" si desea saber en tiempo real quién gana tras el cierre de urna.

## 4. Discusión Técnica y Problemas Abordados

### La naturaleza Stateless de HTTP vs. El estado de la Urna

REST dicta que la comunicación deba ser "stateless" (sin estado). Esto no significa que el servidor no pueda tener base de datos, sino que cada petición HTTP debe contener en sí misma toda la información necesaria para ser procesada sin depender de un "contexto de sesión" almacenado previamente. Nuestro sistema es stateless porque no vincula un usuario a un socket abierto. Sin embargo, para mantener el estado lógico temporal (censo y urna) de la aplicación de votación, se ha hecho uso de estructuras en memoria de Python.

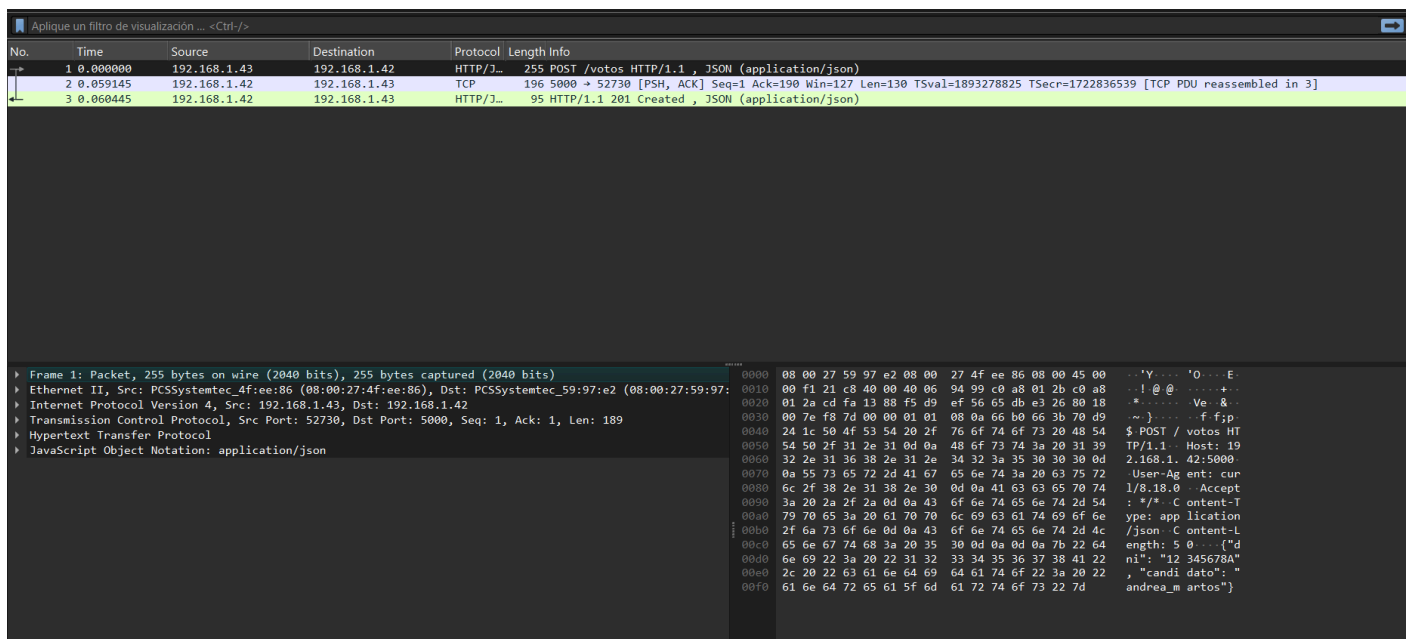
### Diseño de URLs y Transiciones de Estado

Para cambiar el estado de la urna, se descartó el uso de `POST /urna` por no ser idempotente y de un hipotético `POST /cerrar` por violar la pureza de los recursos. La solución adoptada es `PATCH /urna` con cuerpo `{"estado": "cerrada"}`. Esto es semánticamente impecable para modificar un único atributo del recurso principal. Además, la respuesta 200 de este `PATCH` devuelve el balance de resultados final resolviendo los ganadores o el empate sin necesidad de endpoints adicionales.

### Robustez ante Concurrency

FastAPI utiliza ejecución asíncrona basada en bucles de eventos. Aunque el acceso a variables compartidas es parcialmente seguro por diseño (Global Interpreter Lock - GIL), se incorporará un `asyncio.Lock()` en la manipulación crítica de las estructuras, garantizando que cierres de urna o votos paralelos no provoquen lecturas o escrituras sucias durante su evaluación bajo estrés asíncrono.

## 5. Captura Wireshark



En la siguiente captura de Wireshark mostramos la ejecución en red entre nuestras dos máquinas virtuales para comprobar que el sistema funciona correctamente.

Como se puede ver en la primera línea, nuestra máquina cliente (IP 192.168.1.43) envía una petición utilizando el método `POST` al endpoint `/votos` del servidor (IP 192.168.1.42). Dentro de este paquete viaja el payload en formato JSON con los datos del DNI y el candidato elegido. En la segunda línea aparece un paquete TCP de control (ACK) con el que el servidor confirma que le ha llegado la información, y justo después, en la tercera línea, el servidor responde con un código 201 Created para confirmar que el voto se ha registrado con éxito.

Con esta captura demostramos que el intercambio de cabeceras y datos funciona bien y que la API cumple con el principio stateless (sin estado) que pide la arquitectura REST.

## 6. Conclusión Final

La migración del sistema de votación desde un modelo de objetos distribuidos (RMI/Sockets) hacia una arquitectura orientada a servicios web con REST y FastAPI representa un salto cualitativo en términos de interoperabilidad, escalabilidad y estandarización.

Se ha logrado desacoplar completamente la lógica del cliente y del servidor gracias al uso de HTTP y JSON, eliminando las restricciones de lenguaje y plataforma del enfoque anterior. Además, la adopción de principios RESTful puros —mapeando entidades a URLs y acciones a verbos HTTP, y apoyándose rigurosamente en los códigos de estado para la gestión de excepciones y reglas de negocio— ha resultado en una API mucho más limpia, robusta e intuitiva. La inclusión de documentación interactiva autogenerada (Swagger/OpenAPI) y la gestión concurrente segura consolidan un diseño que cumple con las mejores prácticas del desarrollo moderno de aplicaciones en red.